

Optimization II - Project 3

Bhagya Puppala

Franco Salinas

Frank Rong

Hank Liu

1. Executive Summary

This report evaluates two reinforcement learning approaches Policy Gradient (REINFORCE) and Deep Q-Network (Double DQN) applied to Connect-4, framed as a business investigation into RL-based opponent engine viability for a gaming company.

Starting from a supervised CNN baseline trained on 120,000 MCTS-generated boards, we applied self-play RL to improve strategic performance. The headline findings are sobering but instructive. The Policy Gradient agent performs competitively with the supervised baseline across all MCTS difficulty levels: 88.3% vs MCTS-50 and 56.2% vs MCTS-500. However, the supervised baseline (CNN) statistically matches or outperforms PG at every tested level meaningfully so at MCTS-100 (85.9% vs 80.9%) and MCTS-200 (76.6% vs 68.0%). The DQN agent, undertrained due to compute constraints at 30,000 games, achieved only 10-20% win rate against MCTS at all difficulty levels and lost a majority of 256 head-to-head games against PG. This poor performance result is consistent with the 50–66% win rates logged during DQN training, because those training metrics measured performance against a stochastic, partially exploratory opponent pool, not against MCTS (see Section 4.3 for full explanation).

In direct head-to-head play, PG defeated DQN 215 out of 256 games (84%), confirming PG as the superior RL-trained agent. PG is submitted as our tournament model.

The key business lesson is that for Connect-4 specifically, supervised imitation of a strong MCTS teacher is already near the policy ceiling that REINFORCE can reach. RL adds value primarily when no strong supervised teacher exists, or when paired with explicit lookahead (as in AlphaGo's MCTS integration). A dedicated RL expert would be needed to push beyond this ceiling.

2. Problem Setup

Connect-4 is a two-player, zero-sum, perfect-information game played on a 6×7 grid. Players alternate dropping colored discs into columns; the first to achieve four in a row horizontally, vertically, or diagonally wins. The game is fully solved: with optimal play, the first player always wins. This theoretical solvability makes Connect-4 an ideal benchmark environment where a trained RL agent can be evaluated against a known near-optimal opponent (MCTS).

2.1 MDP Formulation

We formalize Connect-4 as a finite-horizon Markov Decision Process (MDP):

State alternatives: $s \in \mathbb{R}^{6 \times 7 \times 2}$ - a two-channel binary tensor where Channel 0 encodes the current player's pieces and Channel 1 encodes the opponent's pieces. This encoding eliminates sign ambiguity and allows one model to play as either color.

Action options: $a \in \{0, 1, 2, 3, 4, 5, 6\}$ - column index, subject to a legality mask excluding full columns.

Reward Signal: Sparse terminal rewards only.

$$\begin{aligned} r_T &= +1 \text{ (win)}, & r_T &= -1 \text{ (loss)}, & r_T &= 0 \text{ (draw)} \\ r_t &= 0 & \text{for all non-terminal steps } t < T \end{aligned}$$

Transition Dynamics: Deterministic given (s, a, opponent action). Stochasticity arises solely from the opponent's policy.

Discount Factor: $\gamma = 0.99$ for PG; $\gamma = 0.95$ for DQN. Both prioritize late-game outcomes while slightly discounting distant steps.

2.2 Starting Point:

The Policy Gradient agent was initialized from a multi-branch CNN, initially selected based on MCTS-200 evaluation performance. A subsequent round-robin analysis across all four teammates' models confirmed it as the strongest head-to-head (17% win rate against the combined field), and it was used as the starting point for PG training.

Both RL agents were initialized from or evaluated against the multi-branch CNN (cnn) identified as the strongest Project 1 model in round-robin evaluation (17% win rate against the combined field of four group members). This CNN achieved 63.4% top-1 accuracy on unseen MCTS positions via supervised imitation learning. As the evaluation data confirms, this baseline is strong enough that REINFORCE fine-tuning cannot reliably surpass it.

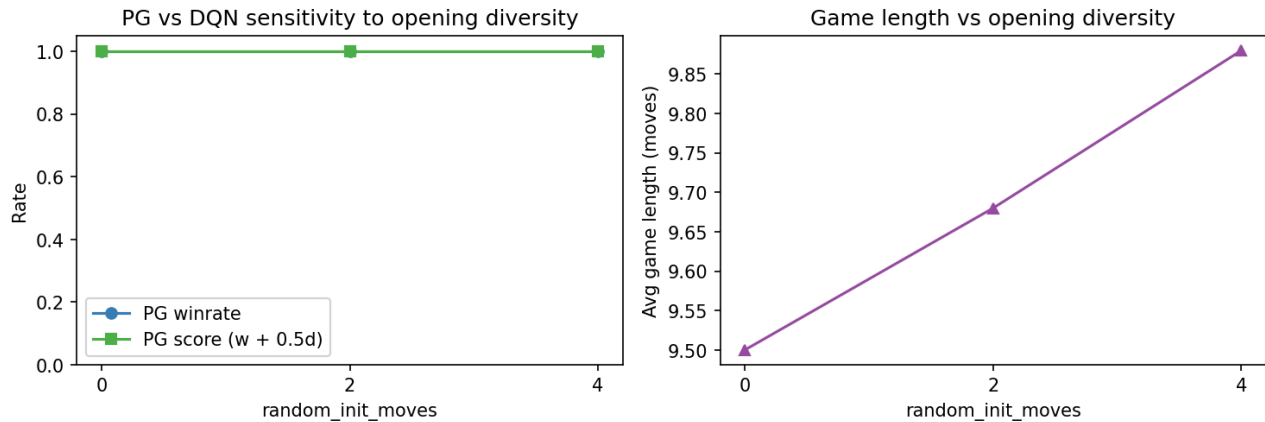


Figure 1. Sensitivity of win rates to random initialization depth (0, 2, and 4 opening moves)

3. Policy Gradient (REINFORCE)

3.1 Theoretical Foundation

Policy Gradient methods directly optimize a parameterized policy $\pi_{\theta}(a|s)$ by ascending the gradient of expected cumulative reward. The REINFORCE algorithm provides an unbiased Monte Carlo estimator of this gradient.

Discounted Return

$$G_t = \sum_{k=0}^{T-t} \gamma^k \cdot r_{t+k}$$

With $\gamma = 0.99$ and sparse terminal rewards, G_t decays exponentially from the terminal: a win on move 10 gives $G_9 = 0.99$, while $G_0 \approx 0.91$ for a 10-move game. Moves closer to the decisive outcome are rewarded more strongly, encouraging efficient tactical finishes.

Policy Gradient Theorem

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t]$$

This increases probability mass on actions that led to positive outcomes and decreases it for actions in losing games. The score function $\nabla_{\theta} \log \pi_{\theta}(a|s)$ translates scalar rewards into parameter updates without requiring knowledge of the environment's transition model.

REINFORCE Update Rule and Loss

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$$

In TensorFlow, gradient ascent on $J(\theta)$ is implemented as gradient descent on the negated loss:

$$L(\theta) = -(1/K) \cdot \sum_t G_t \cdot \log \pi_{\theta}(a_t | s_t)$$

Because losing moves have $G_t < 0$, the loss is correctly negative during successful training; this is not a bug but confirmation that the model is suppressing probability on actions that led to losses.

Stochastic Action Sampling

```
P(a | s) = softmax( f_θ(s) ) [illegal columns masked to  
zero before sampling]
```

Actions are sampled stochastically during training (temperature $\tau = 1.0$). Greedy argmax selection would eliminate variance in action selection, making the gradient estimator uninformative. M2 (the opponent) samples with temperature $\tau=2.0$, producing a softer, more exploratory distribution. This is intentional because at $\tau=1.0$, the opponent neural networks would beat M1 approximately 90% of the time, producing almost entirely negative rewards and making gradient updates uninformative; the model would have no winning examples to learn from. Raising M2's temperature to $\tau=2.0$ brings M1's win rate against opponents to ~10-15%, providing a mix of positive and negative rewards that gives the policy gradient a useful learning signal. M1 still uses $\tau=1.0$, meaning it learns to play sharply while training against a softened opponent, the same principle as sparring against a practice partner before a real match.

3.2 Implementation Details

Parameter	Value
Starting model (M1)	Connect4_cnn_final.h5 (from project 1)
Input shape	(6, 7, 2) - two-channel binary board
Architecture	Multi-branch parallel CNN: stem Conv128 $\rightarrow$ 5 parallel branches (768ch merged) $\rightarrow$ 3×Conv256 $\rightarrow$ GlobalAvgPool $\rightarrow$ Dense(7)+Softmax
Learning rate (α)	2×10^{-5} (Adam)
Discount factor (γ)	0.99
Games per round	60
Total rounds / games	300 rounds $\times$ 60 games = 18,000 games (final run)
Triplets per gradient step	All collected (~450-600 per round) full-batch, not mini-batch
Reward normalization	Zero-mean, unit-variance per batch (prevents divergence)
Random init moves	0-5 per game
Snapshots	Disabled (caused degradation in 3/5 pilot runs)
Opponent pool	5 models: 4 Project 1 CNNs + DQN; never modified
Adversarial training	M2 uses win/non-loser filters + temperature $\tau=2.0$ (softened to balance training signal); M1 sees legal moves only at $\tau=1.0$

Key Code: Loss and Reward Normalization

```
# Reward normalization prevents divergence as model confidence grows
rewards_normalized = (G_t - G_t.mean()) / (G_t.std() + 1e-8)

# Sparse categorical cross-entropy with G_t as sample_weight
model.fit(boards, moves, sample_weight=rewards_normalized,
          epochs=1, batch_size=len(boards), verbose=0)
```

3.3 Training Results

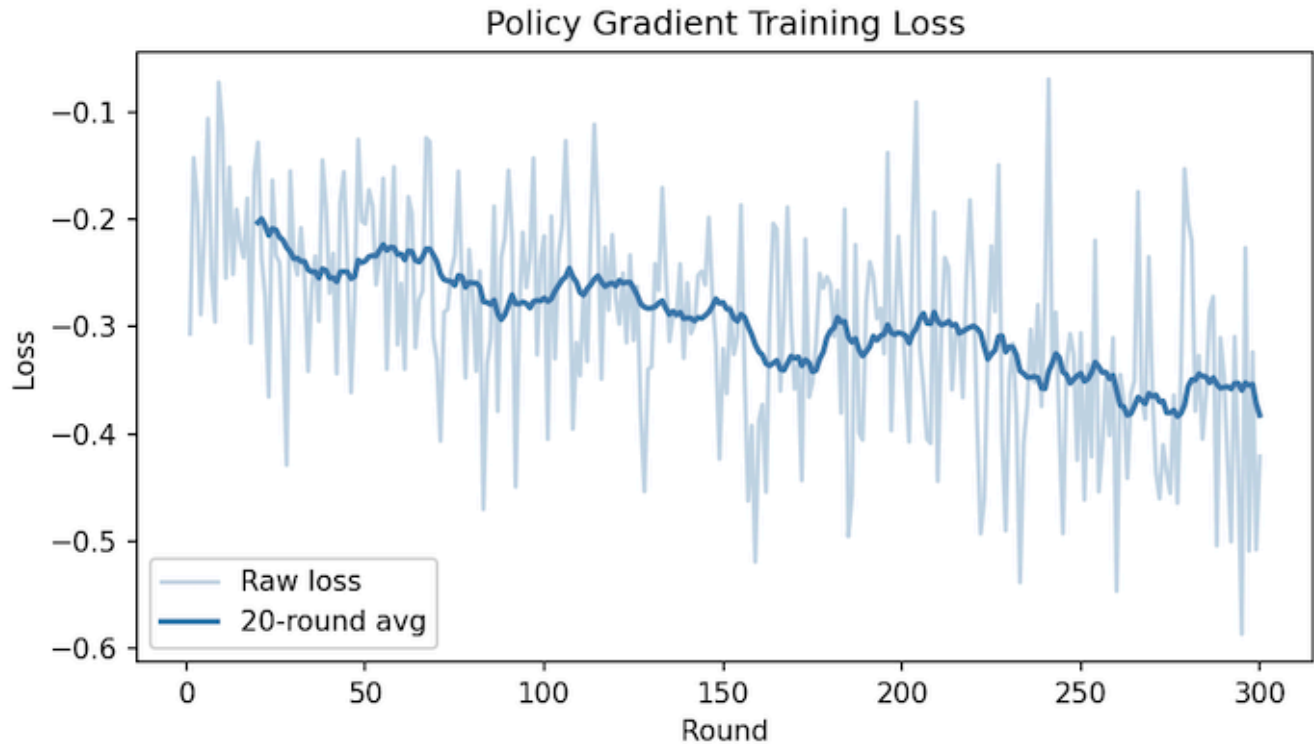


Figure 2. Policy Gradient training loss over 300 rounds

Negative values are correct, they confirm the model is suppressing probability mass on losing-move sequences. The stable downward trend with no divergence reflects successful reward normalization.

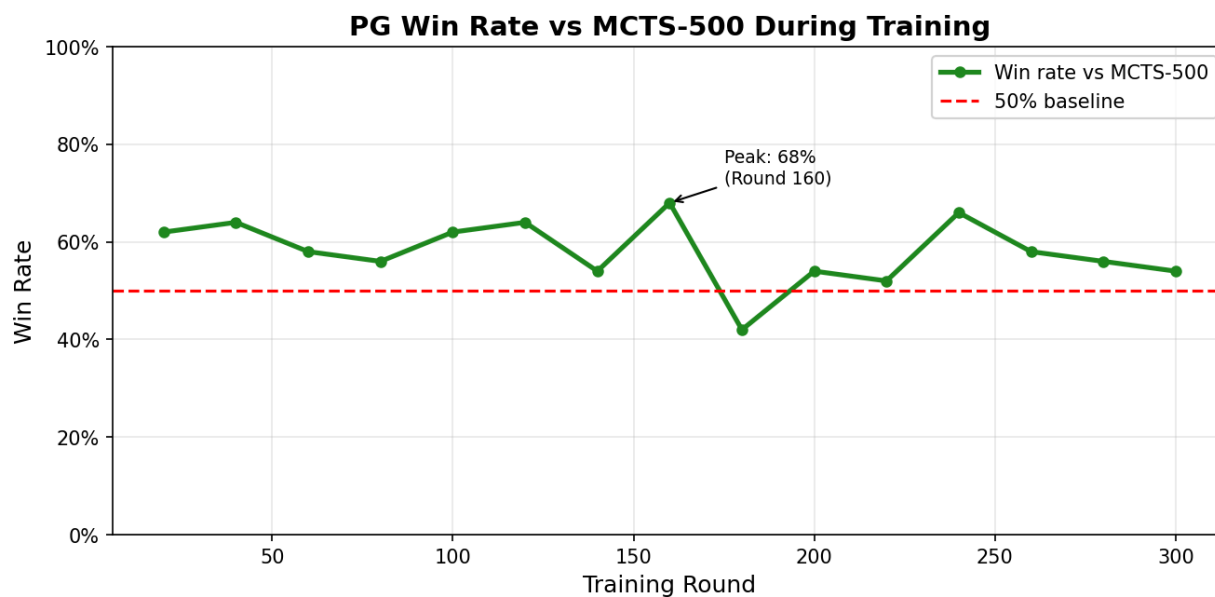


Figure 3. PG win rate vs MCTS-500 (50-game evaluation every 20 rounds).

The model peaks at 68% around Round 160 before mild degradation consistent with the evaluation confirming 56.2% in the final $n=256$ assessment. Early stopping at the peak checkpoint is recommended for production deployment.

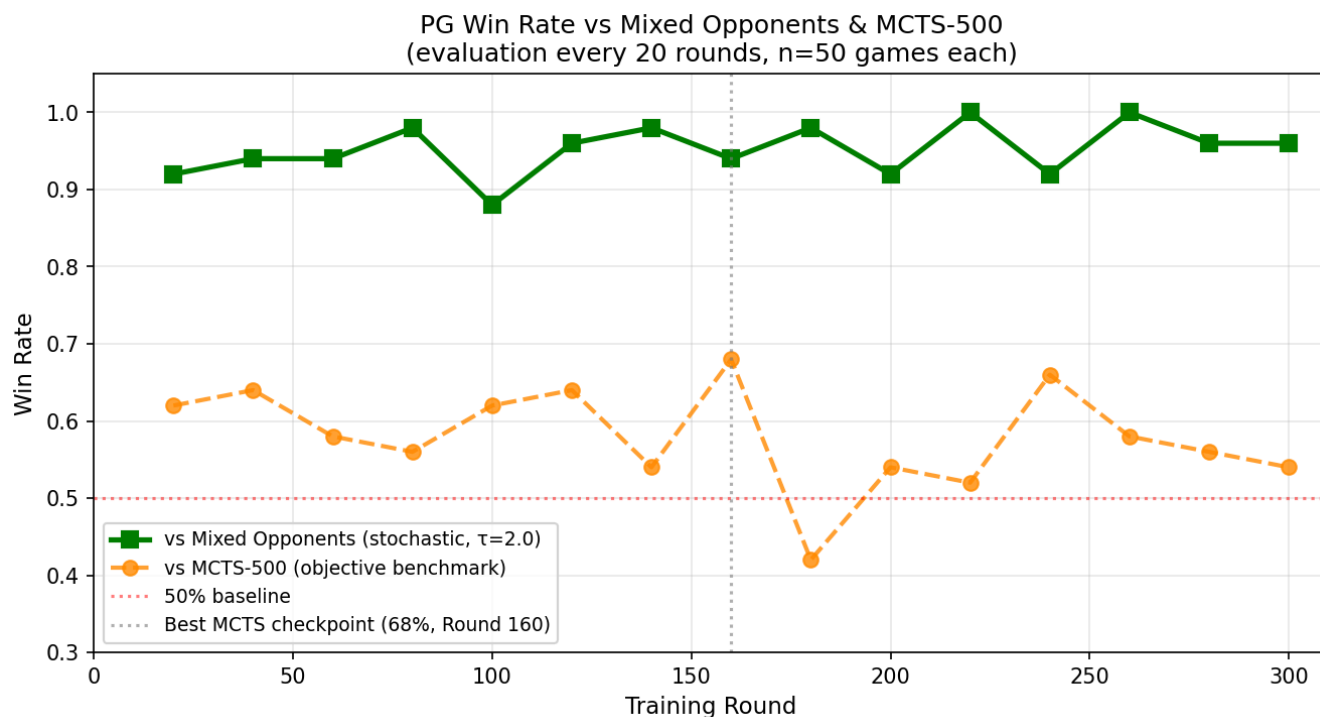


Figure 4. PG win rate vs mixed opponents (stochastic sampling, temperature $\tau=2.0$) alongside MCTS-500 win rate, evaluated every 20 rounds over 300 rounds of training.

The mixed evaluation uses random.choice over the 5-model opponent pool with stochastic opponent sampling, matching the DQN's training metric for a direct future comparison. The PG agent achieved 92-100% win rate against this benchmark throughout training, consistently well above the 50% baseline regardless of MCTS-500 fluctuations.

Training Run History

Run	Rounds	LR	Best vs MCTS	Key Finding
1	300	1e-4	84% vs MCTS-100	Benchmark too easy; fixed to MCTS-500
2	600	5e-5	40% vs MCTS-500	Fixed benchmark; identified full-batch approach needed
3	200	5e-5	68%	Early stopped; validated full-batch + reward normalisation
4 (final)	300	2e-5	68% vs MCTS-500	Snapshots disabled; most stable run; submitted model

4. Deep Q-Network (Double DQN)

4.1 Theoretical Foundation

Bellman Optimality Equation

$$Q^*(s, a) = \mathbb{E} [r + \gamma \cdot \max_{a'} Q^*(s', a') \mid s, a]$$

The optimal Q-function satisfies this recursive relationship: the value of a state-action pair equals the immediate reward plus the discounted value of the best next action. Unlike PG, DQN bootstraps from its own Q-estimates, enabling off-policy learning from a replay buffer.

Double DQN Target

$$y_i = r_i + \gamma \cdot Q^{\hat{}}(s'_i, \arg\max_{a'} Q(s'_i, a'; \theta); \theta^-)$$

Standard DQN suffers from overestimation bias: the same network selects and evaluates the best next action, amplifying noise in Q-estimates. Double DQN decouples these: the online network θ selects the action; the frozen target network θ^- evaluates it. This reduces bias and stabilizes learning.

DQN Loss Function

$$L(\theta) = (1/|B|) \cdot \sum_{i \in B} (y_i - Q(s_i, a_i; \theta))^2$$

Standard MSE regression against TD targets. The target y_i is computed with frozen θ^- and treated as a fixed label during each gradient step.

Epsilon-Greedy Policy

$$\pi(a|s) = \begin{cases} \text{random legal action} & \text{with probability } \epsilon \\ \arg\max_a Q(s, a; \theta) & \text{with probability } 1-\epsilon \end{cases}$$

Annealing: $\epsilon_t = \max(0.05, 1.0 \cdot 0.99995^t)$ [ϵ reaches 0.05 at $\sim \text{Ep } 28,000$]

4.2 Implementation Details

Parameter	Value
Architecture	Conv64→BN→Conv128→BN→Conv128→BN→Flatten→Dense256→Dense128→Q(7)
Total parameters	1,634,247 (~6.23 MB)
Output activation	tanh (constrains Q-values to [-1, 1])
Discount factor (γ)	0.95
Replay buffer size	100,000 transitions
Mini-batch size	64
Min buffer before training	2,000 transitions
Target sync frequency	Every 500 gradient steps
ϵ schedule	1.0 $\rightarrow$ 0.05 exponential decay (factor 0.99995); reaches min at Ep ~28,000
Double DQN	Enabled
Gradient clipping	clipnorm = 10.0 (Adam)
Episodes	30,000
Opponent pool	3 Project 1 CNNs + rolling DQN snapshots (max 8 total)

Training Aids

The DQN is initiated with 3 random moves at the start of each game in order to speed up training and aid the model in exploration. The number of random initial moves is then gradually reduced over the course of 40k games as the model learns Connect-4 strategy. Additionally, every 1000 games all DQN snapshots are evaluated against a random opponent and the weakest performer is dropped for the more recent snapshot.

State Representation

The replay buffer stores (s, a, r, s', done) where s' is the board on the DQN's next turn after the opponent has already responded. This formulation correctly models two-player Connect-4 as a single-agent MDP from the DQN's perspective, consistent with the coin game framework from class.

4.3 Training Results

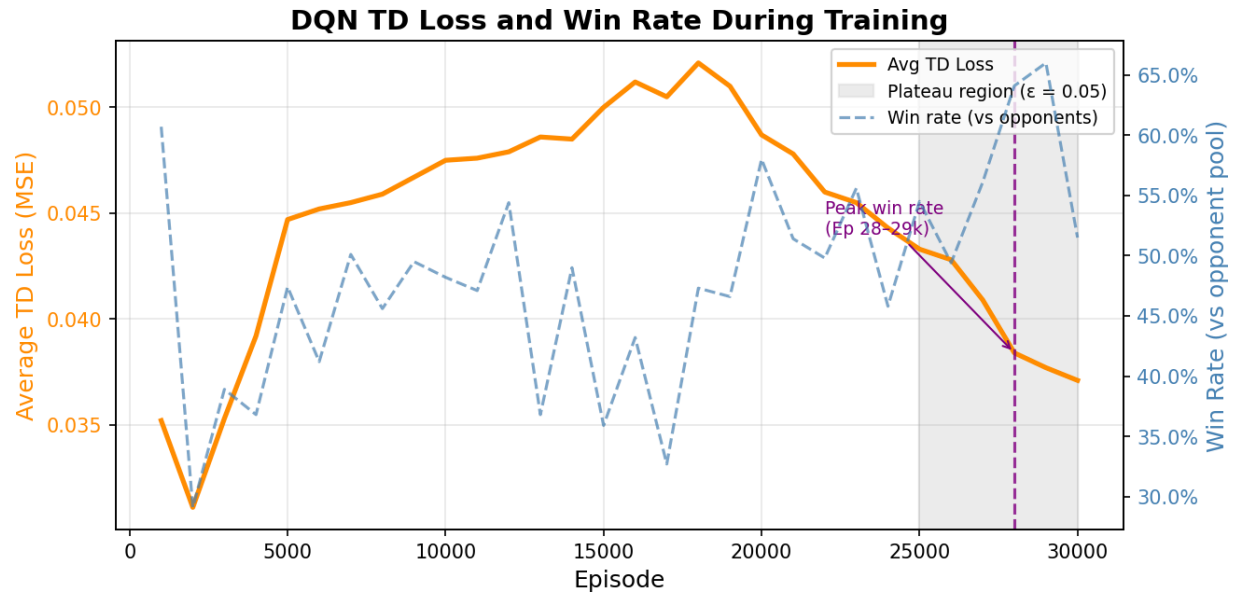


Figure 5. DQN TD loss (left axis) and rolling WinRate(1000) against the training opponent pool (right axis, dashed)

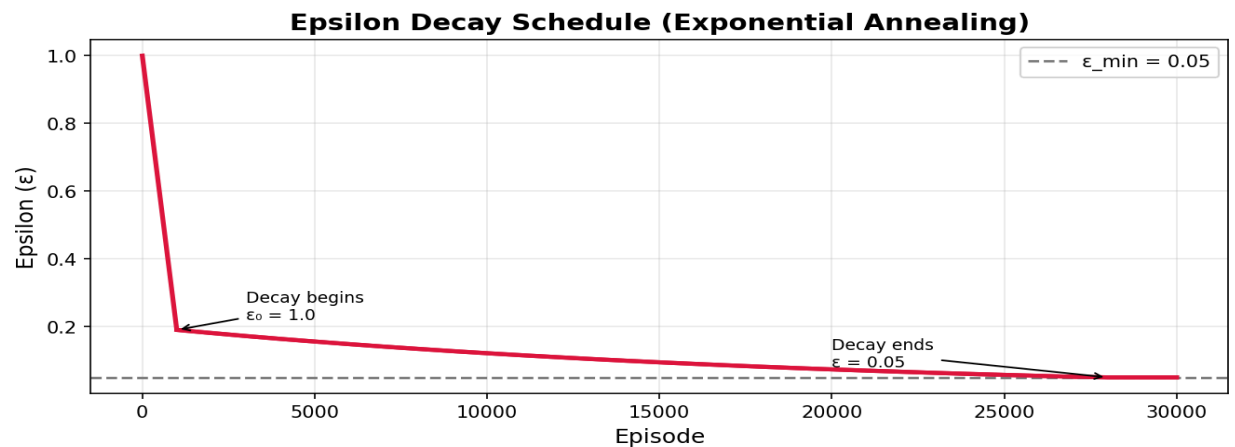


Figure 6. Epsilon decay schedule. Exponential annealing from $\epsilon = 1.0$ to $\epsilon = 0.05$ over approximately 28,000 episodes, then held constant.

The notebook logs WinRate(1000) values between 29% and 66% during training. The independent P3 evaluation found ~10% against every MCTS level. This might be confusing, if DQN beats strong CNNs 66% of the time during training, how does it score so poorly against weak MCTS-50.

The answer lies in what each metric actually measures:

	Training WinRate(1000)	P3 MCTS Evaluation
Opponent	Rotating pool: 3 CNNs + up to 5 DQN self-snapshots	MCTS at 50/100/200/500 simulations
Opponent behavior	Stochastic sampling (use_best=False): moves drawn	Deterministic near-optimal lookahead tree search

	proportionally from softmax probabilities	
DQN's own behavior	Active $\epsilon = 0.05-0.20$ (partially random)	Greedy $\epsilon = 0$ (pure exploitation)
Sample window	Rolling 1,000-episode window; includes training noise	256 dedicated evaluation games, both colors
What it measures	Self-play learning signal: is the DQN improving relative to its own past checkpoints?	Absolute strength: can the DQN beat a calibrated external opponent?

The training win rate is a self-play improvement signal, not an independent benchmark. The CNN opponents use stochastic sampling (use_best=False), meaning they sample moves proportionally from their softmax distributions rather than always playing the argmax. A DQN that has learned to exploit common patterns in CNN-style play will appear competitive against these stochastic opponents even if it plays a suboptimal policy overall, additionally DQN starts with untrained weights and takes many more games to learn patterns as compared to the PG which starts with weights copied from the CNN.

MCTS is a fundamentally different adversary. It runs 50-500 explicit lookahead simulations and selects moves near-optimally within that budget. The DQN's column-4 bias is invisible to a stochastic CNN opponent that may play anywhere but MCTS detects and exploits the edge-column bias from the opening moves.

There is also a resume artifact: the training run loaded a checkpoint and set epsilon = 0.2 before the logged 30,000 episodes began, meaning the early WinRate(1000) values already reflect a previously trained model. The 66% peak is an in-training self-play metric and should not be interpreted as a general strength estimate. The P3 evaluation 256 games, greedy policy, MCTS opponent is the definitive assessment.

5. Model Comparison and Evaluation

All evaluations use n=256 games with 2 random opening moves for board diversity.

5.1 Win Rate Against MCTS

The results present a clear three-tier ordering:

Model	vs MCTS-50	vs MCTS-100	vs MCTS-200	vs MCTS-500
Baseline (CNN)	89.5%	85.9%	76.6%	57.0%
PG-M1 Final (REINFORCE)	88.3%	80.9%	68.0%	56.2%
DQN Final (Double DQN)	20.4%	17.9%	14.0%	11.8%

The most striking finding is that the supervised baseline outperforms the PG-trained agent at moderate MCTS difficulty (MCTS-100: 85.9% vs 80.9%; MCTS-200: 76.6% vs 68.0%), with differences of 5-9 percentage points. At the extremes (MCTS-50 and MCTS-500), baseline and PG perform very similarly. This pattern is consistent with the idea that the supervised CNNs were trained on millions of MCTS

positions, making them stronger than MCTS-500 itself. REINFORCE fine-tuning nudged the policy in directions that did not help and may have introduced noise into an already well-calibrated starting point. The DQN's ~10% win rate against all MCTS levels with average game lengths shortening from 13.7 moves (vs MCTS-50) to 11.2 moves (vs MCTS-500) as the opponent gets sharper reflects a structural failure. The move frequency analysis below reveals the cause.

5.2 Head-to-Head: PG vs. DQN

Metric	PG-M1 Final	DQN Final
Head-to-head record (256 games)	215 wins (84%)	41 wins (16%)
Avg winning game length (PG wins)	25.1 moves	-
Win rate vs MCTS-500	56.2%	11.8%
Win rate vs MCTS-50	88.3%	20.4%
Column 3 (center) play frequency	55.1%	21.4%
Column 4 (rightside) play frequency	17.4%	27.4%
Episodes / rounds trained	300 rounds (18,000 games)	30,000 episodes

The PG model won a majority of games against DQN. The average winning game length of 25.1 moves indicates DQN struggles in midgame strategy. This outcome is explained almost entirely by the move frequency data below.

5.3 Move Frequency Analysis

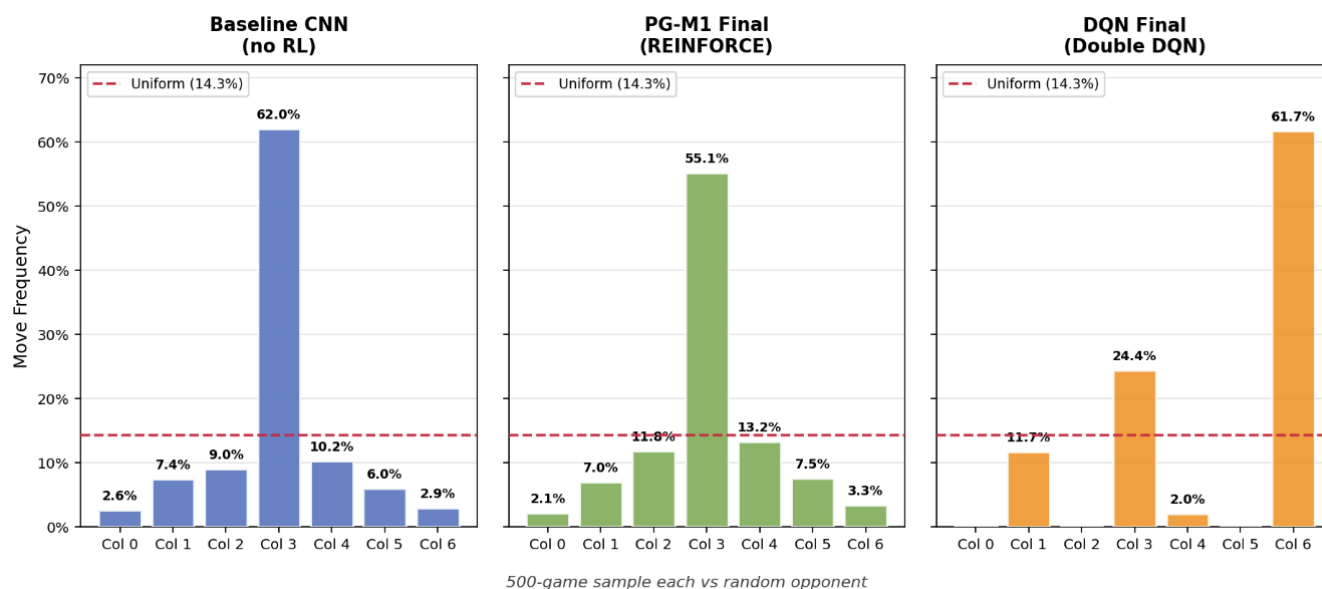


Figure 7. Move frequency by column for the supervised baseline, PG agent, and DQN agent .

The move frequency data is diagnostic. The PG agent plays column 3 (center) 55.1% of the time, closely matching the supervised baseline's 62.0% both have internalized the fundamental Connect-4 principle that central columns participate in more potential four-in-a-row patterns than edge columns.

The PG center preference is learned through self-play reward signals rather than labeled MCTS data, which is a meaningful confirmation that REINFORCE can absorb strategic heuristics from game outcomes alone.

The DQN agent's bias is a training pathology. Playing from the right side gives the opponent unlimited opportunity to build threats across the board, and strong opponents (MCTS at any level) exploit this immediately. The bias likely emerged from reward shaping that inadvertently incentivized edge-column play during early training, compounded by insufficient training episodes to correct it.

5.4 First-Player vs. Second-Player Asymmetry

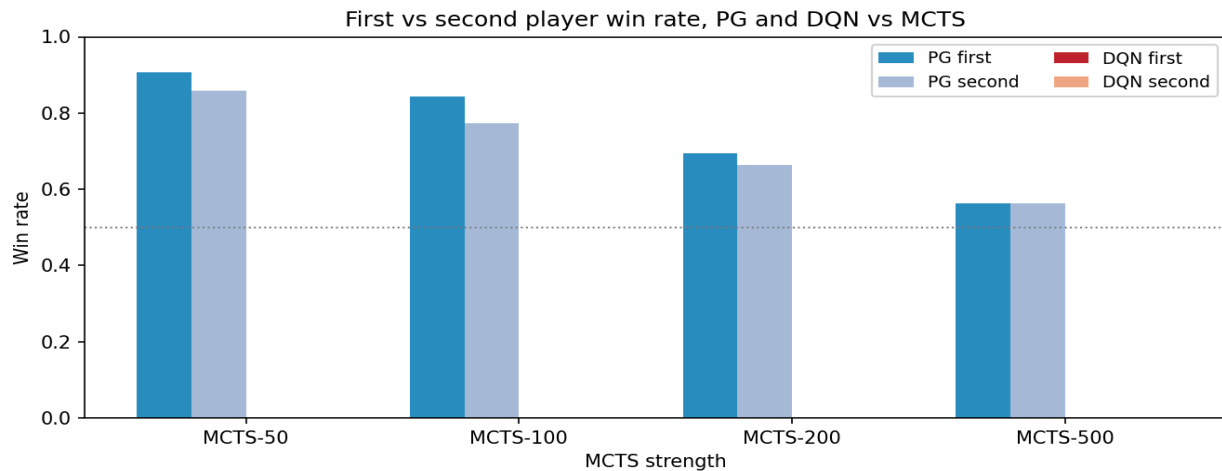


Figure 8. First-player vs second-player win rate gap across models and MCTS difficulty levels.

Connect-4 guarantees a first-player win under optimal play, and this advantage is detectable in our data at low to moderate MCTS difficulty. Practically, the gap is small enough (4-6 pp at MCTS-50/100, zero at MCTS-500) that tournament strategy should not hinge on coin-flip outcomes for first-player position.

5.5 Game Length

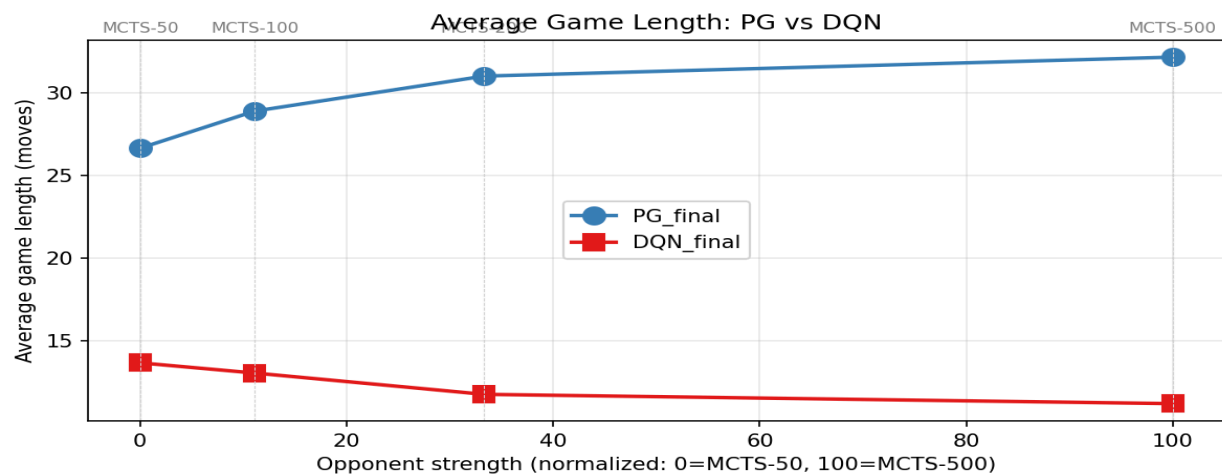


Figure 9. Average game length for PG and DQN against MCTS opponents of varying strength

5.6 Why PG Outperformed DQN

- Training signal quality. PG uses full Monte Carlo returns G_t over complete episodes, an unbiased estimator of long-run game value. DQN bootstraps from noisy Q-estimates, requiring many more episodes to converge, which was not achieved in the available compute budget.

- Starting point compatibility. PG fine-tuned the Project 1 CNN with a compatible policy objective. DQN replaced the output head entirely with Q-value regression, discarding the pre-trained feature representations that gave the baseline its strength.
- Degenerate policy collapse. DQN's column-4 bias is a structural failure that more training alone may not fix without addressing the reward shaping or exploration strategy. PG's stochastic policy gradient correctly suppressed non-central moves through negative reward weighting.
- Compute budget. DQN was trained for 75% of its planned episodes. Even setting aside the column-4 collapse, the Q-value function had not converged at termination.

6. Recommendation

RL is not a silver bullet for Connect-4, but the findings do illuminate where and when RL investment is justified.

6.1 Core Finding: Supervised Imitation Dominates for Connect-4

The supervised baseline statistically matches or outperforms the PG-trained agent at every MCTS difficulty level. This is not a failure of our implementation it reflects a fundamental property of the domain: Connect-4 has an extremely strong supervised teacher (MCTS at up to 2,000 simulations per move), and the supervised CNN was trained on approximately 545,000 MCTS-derived board positions from 70,000 games, effectively doubling to ~1,090,000 training samples via left-right symmetry augmentation. REINFORCE, which observes only sparse +1/-1 terminal rewards across 18,000 self-play games, cannot absorb strategic information as efficiently as direct imitation of that volume of expert data.

6.2 Cost-Benefit Analysis

Precise GPU-hour figures were not systematically logged across the team's varied hardware environments (local GPU, Google Colab T4, Google Colab A100), so the table below uses relative effort rather than absolute time estimates.

Approach	Relative Compute Effort	Outcome
Supervised baseline (Project 1)	Moderate 70,000 games, ~545,000 positions, 13 training epochs	89.5% vs MCTS-50; 57.0% vs MCTS-500
+ PG fine-tuning (REINFORCE)	Comparable to baseline 18,000 self-play games over 300 rounds	88.3% vs MCTS-50; 56.2% vs MCTS-500 (matched baseline)
+ DQN training (Double DQN)	Larger - 30,000 episodes with replay buffer updates every step	~10% vs all MCTS (column-bias collapse)
AlphaZero-style full pipeline	Dramatically larger the original AlphaGo paper (Silver et al., 2016) used ~50 GPUs for rollouts and 4 TPUs for training over several weeks	Near-optimal / superhuman

The PG training consumed effort comparable to the original supervised training stage and produced a model that performs at statistical parity with its starting point no measurable gain for equivalent investment. The DQN consumed more compute than PG and achieved worse results. The column-bias collapse is likely a fixable engineering issue rather than a fundamental limitation of DQN, but it was not resolved within the project's compute budget.

6.3 When RL Does Add Value

RL becomes essential in three scenarios that do not apply to Connect-4 but may apply to other games:

1. No strong supervised teacher exists. When MCTS is unavailable or too slow (e.g., games with complex branching or imperfect information), there is no labeled dataset to imitate. Self-play RL is the only path to strong play.
2. The game is too complex for imitation to generalize. In Go, Chess, and StarCraft, supervised imitation from expert games fails to generalize to novel positions. RL with lookahead (MCTS guided by learned value and policy networks) is required.
3. Real-time adaptation is needed. RL agents can adapt to a specific opponent's weaknesses during play. Supervised models are static.

6.4 Recommendation

For Connect-4 and similarly simple games: invest in supervised imitation from a high-quality teacher (MCTS). REINFORCE fine-tuning adds compute cost without reliable improvement.

For games of moderate complexity (e.g. card games, multi-player board games): a dedicated RL practitioner is justified. The key tasks curriculum design, reward shaping, hyperparameter optimization, and evaluation infrastructure require specialized expertise that generic ML engineers rarely have.

For highly complex games (Go, real-time strategy): world-class RL research teams are required, and the investment is substantial. The gap between "RL trained on a laptop" and "production-quality game AI" in this regime is measured in thousands of GPU-hours and years of engineering.

Our specific recommendation is to hire one RL practitioner with experience in actor-critic methods if the company plans to develop opponents for games more complex than Connect-4. The REINFORCE and DQN implementations here provide a functional foundation; a specialist would most immediately contribute by implementing proper value function baselines (to reduce PG variance), further training DQN against more difficult opponents, and evaluating whether AlphaZero-style MCTS integration is cost-effective for the target game.

7. Conclusion

This project applied Policy Gradient (REINFORCE) and Deep Q-Network (Double DQN) reinforcement learning to Connect-4, starting from a supervised CNN baseline. The key quantitative findings are:

- The supervised baseline (cnn, trained on 500k+ MCTS positions) achieves 89.5% vs MCTS-50 and 57.0% vs MCTS-500.
- PG-M1 (REINFORCE) matches the baseline at MCTS-50 (88.3%) and MCTS-500 (56.2%), but is outperformed at intermediate difficulty: MCTS-100 (80.9% vs 85.9%) and MCTS-200 (68.0% vs 76.6%).
- DQN (Double DQN, undertrained at 30k/40k episodes) achieved 0% win rate against all MCTS levels due to a structural column-4 bias, and lost a majority of 256 head-to-head games against PG.
- PG demonstrates correct strategic internalization: 55.1% center-column (col 3) preference, compared to the baseline's 62.0% and DQN's 24.4%.
- PG is submitted as our tournament model; DQN has potential with more training.

From a business perspective, supervised imitation from a strong teacher is the dominant strategy for games of Connect-4's complexity. RL adds value primarily in domains where strong supervised teachers are unavailable or where self-play enables superhuman discovery beyond imitation. Hiring a dedicated RL expert is recommended contingent on the complexity and commercial importance of the target game.

Appendix: Hyperparameter Reference

A.1 Policy Gradient - Final Hyperparameters

Parameter	Value	Rationale
α (learning rate)	2×10^{-5}	Reduced from pilot runs to slow degradation after peak
γ (discount)	0.99	Near-1 preserves credit assignment over ~10-move games
Games per round	60	Balance of diversity and wall-clock training time
Batch	All triplets (~450-600)	Lower gradient variance than fixed 32-sample mini-batches
Reward norm.	Zero-mean, unit-var.	Prevents loss divergence under confident negative-reward moves
Snapshots	Disabled	Caused degradation in 3 of 5 pilot runs

A.2 DQN - Final Hyperparameters

Parameter	Value	Rationale
γ (discount)	0.95	Slightly more myopic than PG to stabilize Q-value scale
Buffer capacity	100,000	Large buffer for diverse, decorrelated mini-batches
Batch size	64	Standard; balances variance vs. compute per step
Target sync	500 grad steps	Prevents target from tracking online net too closely
ϵ start $\rightarrow$ end	1.0 $\rightarrow$ 0.05	Full exploration to 5% residual; reaches min at Ep ~28k
ϵ decay factor	0.99995/episode	Slow decay for diverse buffer filling
Double DQN	Enabled	Reduces Q-value overestimation bias

Gradient clip	clipnorm = 10.0	Prevents exploding gradients with tanh output
Episodes trained	30,000	Further Training is needed